Quantstamp DRAFT

Nayms - OnRe Offer/Redemption Program Spec

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	RWA
Timeline	2025-03-20 through 2025-03-28
Language	Rust
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Source Code	• nayms/onre-app #a4ab643
Auditors	• Paul Clemson Auditing Engineer • Mostafa Yassin Auditing Engineer • István Böhm Auditing Engineer

Documentation quality	Medium
Test quality	Low
Total Findings	6 Fixed: 3 Acknowledged: 2 Mitigated: 1
High severity findings	0
Medium severity findings	0
Low severity findings	3 Fixed: 1 Acknowledged: 1 Mitigated: 1
Undetermined	0

severity findings	
Informational findings	3 Fixed: 2 Acknowledged: 1

Summary of Findings

The Onre protocol is a Solana program built with the Anchor framework which facilitates the sale of real world assets via an offers system. The protocol allows an authorized user to create offers specifying the buy and sell tokens, a sale price and a maximum amount of tokens available to purchase. The user can then choose to fill these orders (completely or partially) by paying the quoted price. The codebase is well engineered and adheres to good security practices. The security of the project would be further improved by strengthening the codebase's test suite, and this audit review recommends is done before the project is deployed. The audit review found a few issues and it is recommended that the team fix each of them.

Fix Review Update: The team fixed, acknowledged or sufficiently mitigated all issues and the suggestions provided by the audit team.

ID	DESCRIPTION	SEVERITY	STATUS
ONRE-1	**Initialization Can Be Frontrun**	• Low	Mitigated
ONRE-2	**Lack of Public Key Validation**	• Low	Fixed
ONRE-3	**Uniqueness of `offer_id` Not Enforced**	• Low	Acknowledged
ONRE-4	**Potential Loss of Precision when Trading From Tokens with Nine Decimals to Six**	• Informational	Acknowledged

ID	DESCRIPTION	SEVERITY	STATUS
ONRE-5	Lack of Error Reporting During Program Initialization	• Informational	Fixed
ONRE-6	Missing Buy Token Amount Validation	• Informational	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

> **Disclaimer**
> Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.

    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: https://github.com/nayms/onre-app Files: anchor/programs/onreapp/src/*

Operational Considerations

- The boss role in the protocol is trusted to ensure the correct tokens are offered at the correct prices.
- The boss is also trusted to open redemption windows for users to liquidate their tokenized assets and to allow ample time for offers to be filled.
- The Associated Token Accounts (ATAs) used within the protocol are expected to have been created prior to interacting with the program.
- KYC checks are handled off-chain, although the team has expressed an intent to move these on-chain in the future.

Key Actors And Their Capabilities

The protocol has one authorized role:

Boss: The program's administrator. This role has the power to create offers and close offers. The role also has the authority to set a new `boss`.

Findings

ONRE-1 Initialization Can Be Frontrun

• Low ⓘ Mitigated

Update
Marked as "Fixed" by the client.
Addressed in: `8b12c969b157c77eb3beceebafca2843d025bb07`.

The client added error messages to spot if front-running occurred. However, front-running is still possible and, if it happens, the deployer will have to do a new deployment, which could add extra-efforts, and users may mistakenly engage with the incorrect program due to front-running. The team should be careful to only make the program address public once a sucessful deployment has been confirmed. For these reasons, we considered that this issue is Mitigated.

File(s) affected: `initialize.rs`

Description: The `instructions::initialize::initialize()` function can be frontrunned after the program is deployed. An attacker might create a state and set themselves as a boss, preventing the real admin from controlling the program.

Recommendation: Consider enforcing a known hard-coded boss key during initialization or use an alternative method to prevent front-running attacks.

ONRE-2 Lack of Public Key Validation

Low Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `df50deff8a59c3170c45a487e1e62cb90ced727a`.

File(s) affected: `set_boss.rs`

Description: The `instructions::set_boss::set_boss()` function does not validate the `new_boss` public key value before setting it, resulting in resetting the boss account identifier to the default public key if it is sent in the transaction. Setting the default value allows another user to reinitialize the program and gain admin-level permission.

Note that this is similar to the lack of zero address checks in Solidity code bases.

Recommendation: Consider validating that the passed pubkey is not Pubkey::default().

ONRE-3 Uniqueness of `offer_id` Not Enforced

Low Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

```
We are aware of this possibility. The offer ids will be stored in
another off chain application and will be kept unique, we are not
checking this on chain. If the offer with an offerId is closed and
made again with the same offer id, we will check in the offchain
application and bind them through timestamps.
```

File(s) affected: `make_offer.rs`

Description: The protocol uses `offer_id` values as critical seed values for deriving Program Derived Addresses (PDAs) for both offer accounts and offer authorities. While the program specification implicitly assumes offer IDs are unique, there is no on-chain mechanism to enforce this uniqueness. Offer IDs can be reused after an offer is closed, which could lead to confusion in the event logs and also issues where previously opened (and then closed) PDA accounts are reopened again.

An additional issue here would be if the `boss` account becomes compromised, the new malicious boss could create an offer specifying a particular token exchange rate and then front run a users attempt to fill that order to create a new order with the same `offer_id` now offering a significantly worse exchange rate to the user.

Recommendation: Consider implementing an on-chain mechanism to ensure offer ID uniqueness. This could be achieved by storing previously used offer IDs in the program state or, more efficiently, by using an incrementing counter for offer ID assignment. Additionally, consider adding a timestamp or version field to offers to provide additional context and help differentiate between offers even if logs are interpreted out of sequence.

ONRE-4
Potential Loss of Precision when Trading From Tokens with Nine Decimals to Six

Informational | Acknowledged

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

```
We acknowledge this and the users can only lose very small amounts
of funds due to precision loss. It cannot be avoided when using
different precision tokens.
```

File(s) affected: `take_offer.rs`

Description: When calculating how many tokens the user should receive in the `calculate_buy_amount()` function, there is a potential for some small amount of precision to be lost when the sell token has more decimals than the buy token. This is important because the team has outlined their intent to use `USDC` (six decimals) in the protocol, while the `ONe` and `rONe` tokens both have nine decimals.

There are two scenarios to be aware of that could cause problems:

1. The user requests to redeem an amount of `rONE` tokens with seven or more decimals.
2. The `USDC` : `ONe` ratio when calling the `take_offer_two()` function causes a normal redemption amount to lose some precision when redeeming a partial amount in `USDC`.

Recommendation: Ensure users are aware of this behaviour to discourage users from specifying redemptions with decimal amounts that could cause precision loss.

ONRE-5
Lack of Error Reporting During Program Initialization

Informational Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `8b12c969b157c77eb3beceebafca2843d025bb07`.

File(s) affected: `initialize.rs`

Description: The `instructions::initialize::initialize()` function does not display an error if `state.boss` is already set. Not returning an error makes it harder to detect if the state was already initialized.

Recommendation: Consider returning an error if the state of the program is already initialized.

ONRE-6
Missing Buy Token Amount Validation

Informational Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `87f0db3aa86a8749fc49afcbad13c4f6e8fa9175`.

File(s) affected: `take_offer.rs`

Description: The `instructions::take_offer::calculate_buy_amount()` function does not return an error if the calculated buy token amount is zero because of integer truncation (rounding). Not checking the return value may lead to unexpected behavior.

Recommendation: Consider checking if the calculated buy token amount is non-zero.

Auditor Suggestions

S1 Lack of Pause Functionality

Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> ```
> We can add this in the future, the boss is going to be a Squads
> vault PDA. Compromising of the boss Squads account is very
> unlikely.
> ```

Description: The program lacks a mechanism to temporarily pause critical operations (e.g., making or taking offers). With a pause feature, the team can halt further transactions if a security issue is discovered until the program is fixed.

Recommendation: Consider adding a pause mechanism controlled by a designated authority (e.g., boss) to disable core functions in an emergency.

S2 Critical Role Transfer Not Following Two-Step Pattern

Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> ```
> Will be added in the future. The boss account is not going to be
> transfered frequently and it is a Squads account.
> ```

Description: The owner of the contracts can call `set_boss()` to transfer the ownership to a new address. If an uncontrollable address is accidentally provided as the new boss address then the contract will no longer have an active boss.

Recommendation: Consider allowing the transfer to happen over two steps, in which the following happens: 1 - The first call sets a `pending_boss` value in storage 2 - This `pending_boss` can then call a second function to accept the role transfer.

This would ensure that the role is always transferred to an address capable of fulfilling the role.

S3 Lack of Offer Active Status Validation

Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> ```
> The take_offer instructions will fail if there are not enough
> tokens. Also new checks of the balances have been added. There is
> going to be an off chain application which monitors the program and
> closing of the offer is going to be made afterwards by a Squads
> account.
> ```

File(s) affected: `take_offer.rs`

Description: An offer is considered closed if its sell token balance reaches zero. However, the `instructions::take_offer::take_offer_one()` and `instructions::take_offer::take_offer_two()` functions do not return an appropriate error if the offer is closed.

Recommendation: Consider returning an error message in the functions if the offer is closed.

S4 Lack of Token Total Amount Validation

Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `7f18e63e9cbbd66621f081faa696ed438bc99228` .

File(s) affected: `make_offer.rs`

Description: The following functions do not validate if the buy and sell token total amount values are non-zero before setting them:

- `instructions::make_offer::make_offer_one()`
- `instructions::make_offer::make_offer_two()`

Recommendation: Consider validating that the total token amounts are non-zero.

S5 Unchecked Arithmetic Operations

Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `89df6e70a48dd03175145434460f4b70921cced8`.

File(s) affected: `take_offer.rs`

Description: The following functions use arithmetic operations that do not use overflow protection when validating the sell token account balance:

`instructions::take_offer::take_offer_one()`
`instructions::take_offer::take_offer_two()`

Recommendation: Consider using checked arithmetic operations instead of unchecked.

S6
Add a Constraint to Ensure `TakeOfferTwo` Struct Cannot Be Used for Offers that Specify only One Token

Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `8b12c969b157c77eb3beceebafca2843d025bb07`.

File(s) affected: `take_offer.rs`

Description: The struct `TakeOfferOne` has the following constraint on the `offer` field to ensure the `offer` provided only has one buy token specified:

```
    #[account(
        constraint = offer.buy_token_mint_2 == system_program::ID @
TakeOfferErrorCode::InvalidTakeOffer
    )]
    pub offer: Account<'info, Offer>,
```

However, there is no equivalent check on the `TakeOfferTwo` struct to ensure two buy tokens are passed to it. While a call with an incorrect offer would likely still fail, having an explicit constraint would be preferred.

Recommendation: For more explict security, consider adding a similar constraint to the `offer` provided in the `TakeOfferTwo` struct to ensure that a valid second buy token has been provided.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

- `d06...d2f ./src/lib.rs`
- `c7b...8d5 ./src/state.rs`
- `652...7f2 ./src/contexts/mod.rs`
- `8d6...f6d ./src/contexts/offer_context.rs`

- `af1...c57 ./src/instructions/initialize.rs`
- `583...e9d ./src/instructions/mod.rs`
- `726...58e ./src/instructions/take_offer.rs`
- `e18...01c ./src/instructions/close_offer.rs`
- `675...760 ./src/instructions/set_boss.rs`
- `ed6...a69 ./src/instructions/make_offer.rs`

Toolset

The notes below outline the setup and steps performed in the process of this audit.

Setup

Tool Setup:

- Cargo Audit 0.21.2

Steps taken to run the tools:

- Navigated to the anchor directory
- Installed via `cargo install cargo-audit`
- Ran `cargo audit`
- Trimmed output of "Dependency tree" for brevity

Cargo Audit identified 2 vulnerabilities in dependencies. Consider reviewing the suggestions and making the recommended changes if the program is impacted.

Automated Analysis

Cargo Audit

```
    Fetching advisory database from
`https://github.com/RustSec/advisory-db.git`
      Loaded 746 security advisories (from
/home/ubuntu/.cargo/advisory-db)
    Updating crates.io index
    Scanning Cargo.lock for vulnerabilities (259 crate dependencies)
Crate:     curve25519-dalek
Version:   3.2.1
Title:     Timing variability in `curve25519-dalek`'s
`Scalar29::sub`/`Scalar52::sub`
Date:      2024-06-18
ID:        RUSTSEC-2024-0344
URL:       https://rustsec.org/advisories/RUSTSEC-2024-0344
Solution:  Upgrade to >=4.1.3
```

```
Dependency tree:
-- trimmed --

Crate:     ed25519-dalek
Version:   1.0.1
Title:     Double Public Key Signing Function Oracle Attack on
`ed25519-dalek`
Date:      2022-06-11
ID:        RUSTSEC-2022-0093
URL:       https://rustsec.org/advisories/RUSTSEC-2022-0093
Solution:  Upgrade to >=2
Dependency tree:
-- trimmed --

Crate:     atty
Version:   0.2.14
Warning:   unmaintained
Title:     `atty` is unmaintained
Date:      2024-09-25
ID:        RUSTSEC-2024-0375
URL:       https://rustsec.org/advisories/RUSTSEC-2024-0375
Dependency tree:
-- trimmed --

Crate:     derivative
Version:   2.2.0
Warning:   unmaintained
Title:     `derivative` is unmaintained; consider using an alternative
Date:      2024-06-26
ID:        RUSTSEC-2024-0388
URL:       https://rustsec.org/advisories/RUSTSEC-2024-0388
Dependency tree:
-- trimmed --

Crate:     paste
Version:   1.0.15
Warning:   unmaintained
Title:     paste - no longer maintained
Date:      2024-10-07
ID:        RUSTSEC-2024-0436
URL:       https://rustsec.org/advisories/RUSTSEC-2024-0436
Dependency tree:
-- trimmed --

Crate:     atty
Version:   0.2.14
Warning:   unsound
Title:     Potential unaligned read
```

```
Date:      2021-07-04
ID:        RUSTSEC-2021-0145
URL:       https://rustsec.org/advisories/RUSTSEC-2021-0145
Dependency tree:
-- trimmed --

Crate:     borsh
Version:   0.9.3
Warning:   unsound
Title:     Parsing borsh messages with ZST which are not-copy/clone is
unsound
Date:      2023-04-12
ID:        RUSTSEC-2023-0033
URL:       https://rustsec.org/advisories/RUSTSEC-2023-0033
Dependency tree:
-- trimmed --

Crate:     borsh
Version:   0.10.3
Warning:   unsound
Title:     Parsing borsh messages with ZST which are not-copy/clone is
unsound
Date:      2023-04-12
ID:        RUSTSEC-2023-0033
URL:       https://rustsec.org/advisories/RUSTSEC-2023-0033
Dependency tree:
-- trimmed--

error: 2 vulnerabilities found!
warning: 6 allowed warnings found
```

Test Suite Results

The test suit can be run by navigating to the `anchor` folder and running the `anchor test` command.

```
 PASS  tests/onreapp.spec.ts (19.239 s)
[0/950]
  onreapp
    ✓ Initialize onre with right boss account (412 ms)
    ✓ Set boss account sets a new boss account (1219 ms)
    ✓ Makes an offer (418 ms)
    ✓ Make offer fails on boss account with non boss signature (510
```

```
ms)
    ✓ Make offer fails on non boss account with boss signature (18 ms)
    ✓ Make offer fails on non boss account with non boss signature (31
ms)
    ✓ Replace an offer (260 ms)
    ✓ Create and take offer (3678 ms)
    ✓ Takes an offer with one buy token successfully (2055 ms)
    ✓ Fails to take offer with one buy token due to exceeding sell
limit (1633 ms)
    ✓ Fails to take offer with one buy token due to invalid buy token
mint (2042 ms)

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        19.298 s
```

Code Coverage

The protocol's test suite covers the majority of happy cases, however more robust simulations of expected actual use of the protocol and thorough testing of edge cases is encouraged.

Changelog

- 2025-03-26 - Initial report
- 2025-04-03 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over $200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with

include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and

non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.